

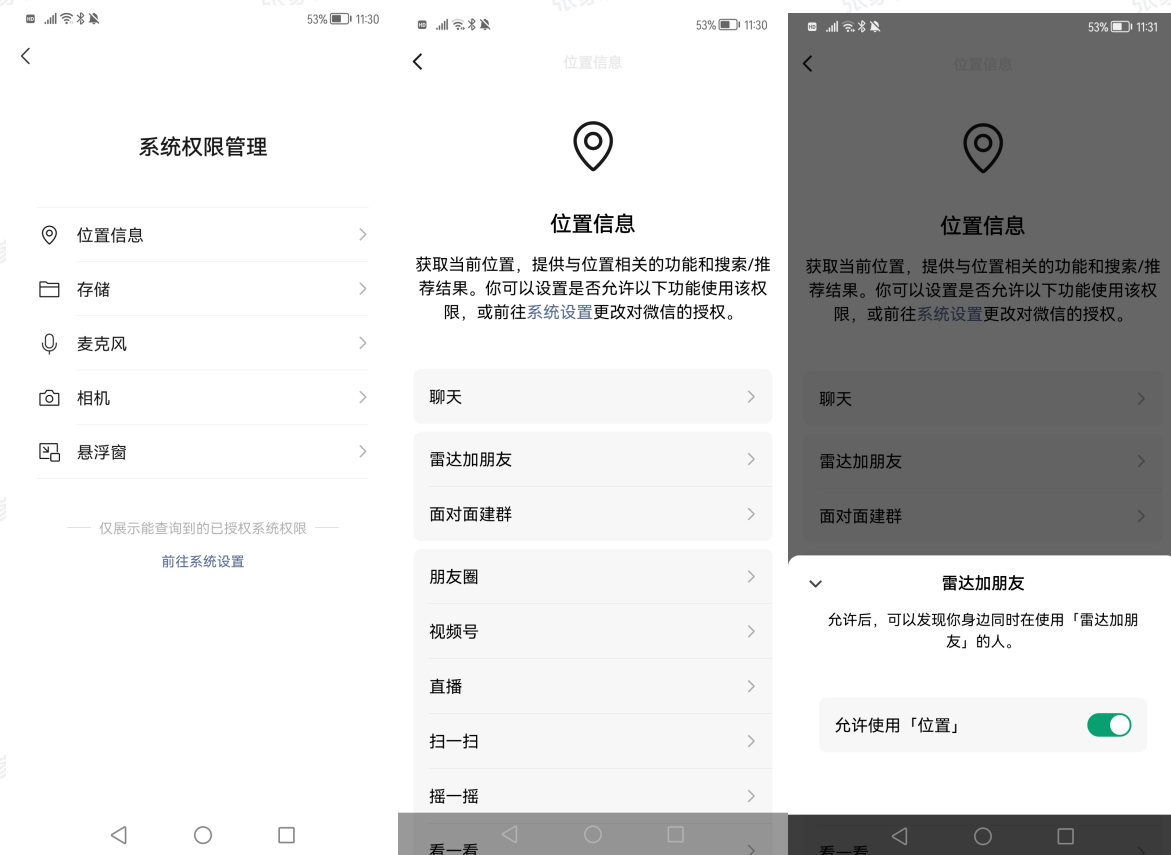
请输入标题

Rightly分模块授权解决方案

1.1 甲方政策解读

目前了解到，关于分模块权限，甲方可能会提以下3点要求：

- 1，隐私协议分模块，进入模式时单独弹窗同意
- 2，APP设置页面中，允许用户按模块管理权限和对应API在各模块的开关状态
- 3，模块首次申请某权限，需弹窗征得用户同意



1.2 整体解决方案

1.2.1 整体方案设计

从政策角度看，APP整体需要具备的能力包括以下三个：

能力1：权限<-> 模块授权关系管理 - 由Rightly提供

- 支持自定义模块与自定义权限的绑定，以支持“**权限管理界面的展示**”
- 支持模块权限同意状态存储，以支持“**记录模块权限同意按钮操作状态**”
- 由Rightly实现，考虑本身业务就要把关系塞进Rightly、业务无需重复存储；另外，多业务间可复用

能力2：模块首次申请权限弹窗 - 业务结合Rightly提供的能力1自己实现

能力3：模块API调用限制 - 由Rightly提供

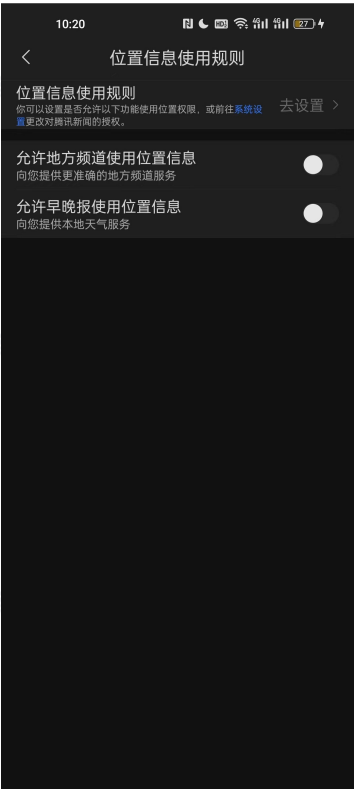
- 支持通过定义权限与API的限制关系，以支持“模块无对应权限时，阻止权限相关的敏感API调用”

能力4：模块隐私同意的管理 - 由Rightly提供（建设中）

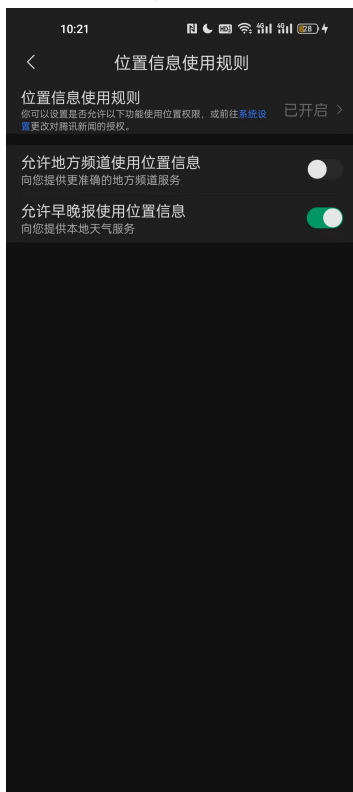
1.2.2 业务接入案例

腾讯新闻

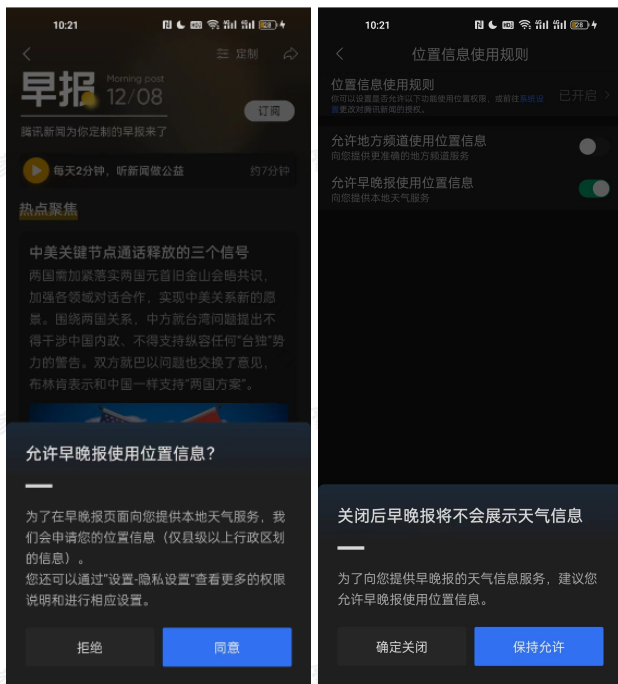
1、分场景权限开关界面



2、无系统权限下，系统权限界面，允许后无感知同意模块，同意后模块开关情况



3、已申请到系统权限的情况下，模块若无权限，需要弹窗申请



1.3 Rightly接入文档

1.3.0 接入版本

0.9.13-rc4

1.3.1 接入文档 - 基础

接入文档将以实现定位权限管控的案例，进行描述。

分模块与兜底规则的关系

Rightly的API兜底规则，支持全局（ updateRuleForApi ）和局部(addSceneRuleForAPI)两种作用域配置。引入分模块后，在模块无权限时所有的API会默认禁用。在模块有权限时，前面所配置的规则不受影响。 这里注意的时如果全局updateRuleForApi配置某API的兜底规则是BACK_BAN_AND_FRONT_BAN，但该API又被纳入权限管理，将会在初始化的时候报错。

目前支持的权限列表

常量类：SplitContant

```
1 // ----- 分场景权限预定义 -----
2 /**
3  * 权限类型 - 定位
4  */
5 public static final String PM_PERMISSION_LOCATION = "perm_location";
6 /**
7  * 权限类型 - 读取通讯录
8  */
9 public static final String PM_PERMISSION_READ_CONTACT = "readContact";
10 /**
11  * 权限类型 - 相册
12  */
13 public static final String PM_PERMISSION_PHOTO = "photo";
14
15 /**
16  * 权限类型 - 摄像头
17  */
18 public static final String PM_PERMISSION_CAMERA = "camera";
19 /**
20  * 权限类型 - 麦克风
21  */
22 public static final String PM_PERMISSION_MICROPHONE = "microphone";
23
24 /**
25  * 权限类型 - 运动健身
26  */
27 public static final String PM_PERMISSION_MOTION = "motion";
28 /**
29  * 权限类型 - 蓝牙
30  */
31 public static final String PM_PERMISSION_BLUETOOTH = "bluetooth";
32 /**
33  * 权限类型 - WIFI
```

配置业务权限

选择预置业务权限Split，限制Location.getAccuracy和

TelephonyManager.requestNetworkScan两个敏感API的执行。在权限未授予或未生效的，API将无法调用。

```
1 // 让部分定位API受到自定义位置权限PERM_LOCATION的控制
2 // 无权限情况下彻底禁止访问，
3 // 拥有权限执行用户配置的策略
4 PMonitor.getConfig()
5     .addPermission(SplitConstant.PM_PERMISSION_LOCATION)
6     .restrictApi(ConstantModel.Location.NAME,
7         ConstantModel.Location.GET_ACCURACY)
8     .submitRule()
```

配置模块并声明权限

前面给API加了权限的限制，那么如何授予权限和让权限生效？这些操作只能通过模块进行，在这得先配置模块，并给模块声明权限。如下配置了WEB，DEVICE共两个模块，并为WEB模块声明了前面定义的位置权限PERM_LOCATION。

```
1 // 定义Web模块，并声明Web模块所需要的权限，让模块与API通过权限绑定起来
2 PMonitor.getConfig()
3     .addModule(BUSINESS_WEB)
4     .usePermission(SplitConstant.PM_PERMISSION_LOCATION)
5     .submitRule()
6 // 定义Device模块，并声明Device模块所需要的权限，让模块与API通过权限绑定起来
7     .addModule(BUSINESS_CHAT)
8     .usePermission((SplitConstant.PM_PERMISSION_READ_PHONE)
9     .submitRule()
```

给模块授权

定义完模块后，得先给模块授权，授权完毕后再调用enter进入模块时，我们才能获得权限，并被允许访问访问这两个定位API Location.getAccuracy和TelephonyManager.requestNetworkScan两个敏感API

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).grantPerm(SplitConstant.PM_PER MISSION_LOCATION)
```

注：Rightly在这一步会存储模块所拥有的权限列表

取消模块授权

在权限管理界面用户选择取消模块权限时，可通过以下API取消模块授权。

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).unGrantPerm(PERM_LOCATION)
```

进入模块（让模块权限生效，API可用）

进入模块时，前面模块已授予权限会生效，权限绑定的API会被允许调用。

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).enter()
```

建议模块进入和退出的时机优先对应界面的进入和退出；如果模块存在需进一步划分权限的情况，考虑将模块时机与权限使用前和权限是使用后进行对应。

注：权限只有在关联模块都已经退出时取消，才能禁止API。

退出模块

注：退出模块时，已授予权限会失效，权限绑定API不允许调用。

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).exit()
```

完全退出模块

由于单个模块的进入和退出都是可以多次的，会存在1.3.2进入退出不匹配导致权限残留。故业务可以在明确退出某模块的场景调用完全退出能力。模块将完全从运行中的模块消失，对应权限也会失效。

案例：QQ浏览器因在网页模块，进入退出不同网页时，出现不对齐情况。在离开网页时调用调用exitAll进行兜底。

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).exitAll()
```

获取所有权限列表

```
1 List<String> allPerms = PMonitor.splitGranter().getAllPermission()
```

获取权限对应的模块

```
1 List<SplitModule> modules =  
  PMonitor.splitGranter().getModulesByPermission(perm)
```

判断模块是否有权限

```
1 PMonitor.splitGranter().module(BUSINESS_WEB).isPermissionGranted(SplitConstant.PM_PERMISSION_LOCATION)
```

1.3.2 接入文档 - 高级

模块绑定UI，锁定生效范围

为避免“模块未正常退出导致权限残留被利用”的问题，可以让模块和界面Activity绑定。若当前Top页面Activity非模块绑定Activity，则模块的权限不会生效。

```
1 PMonitor.splitGranter().module(BUSINESS_DEVICE)
2     .bindUI("MainActivity")
```

模块允许后台调用，穿透Rightly全局兜底策略

在模块已获得权限的情况下，如果全局配置了后台禁用则后台得不到执行。此时可以通过setBackgroundAllowToCall配置后台执行的穿透。

```
1 PMonitor.splitGranter().module(BUSINESS_DEVICE).setBackgroundAllowToCall(true)
```

1.3.3 存在问题和解决方案

1.3.3.1 解决模块交叉问题

Rightly是支持多模块同时进入和退出的。多模块同时在运行时这种现象这里定义为模块交叉。模块交叉可能存在的问题：

1，Module交叉 - 情况1

进入/退出业务模块调用Module#enter/exit打点，但这样Module的生效时间范围太大，可能会被其他模块蹭权限。例：

- 模块A：当前不可见、无权限
- 模块B：当前可见、有权限

模块A此时调用敏感API，可能蹭到模块B权限。

解决：针对调用较多的业务，建议在调用敏感API前后使用Module#enter/exit打点。但加大问题3的出现概率。

2，Module交叉 - 情况2

当UI处于无权限的模块时，有权限的模块可能由于逻辑bug发起调用，导致体验上出现不合理调用情况（如显示定位图标）。例：

- 模块A：当前不可见、有权限
- 模块B：当前可见、无权限

解决1：业务应该避免这种情况发生。为避免出现这种情况

解决2：业务可以采取用UI绑定的形式，来规避兜底，

PMonitor.splitGranter().module(BUSINESS_DEVICE).bindUI("MainActivity")。这样如果当前TopActivity不是MainActivity的时候该模块即使没有退出，它的权限也不会生效。

1.3.3.1 解决模块进入退出不匹配问题

Module#enter/exit业务调用可能不配对，可能导致模块一直无法退出

解决：选择一个时机，调用完全退出API的exitAll，对这类问题进行兜底。

PMonitor.splitGranter().module(BUSINESS_DEVICE).bindUI("MainActivity")。这样如果当前Activity不是MainActivity的时候该模块即使没有退出，它的权限也不会生效。

1.3.4 监控模块问题调用

接入后Rightly将监控上报两类问题：

1、模块无权限调用

用于发现被限制API，未声明模块 或 模块无权限时的调用

2、模块隐私同意前调用（建设中）

用于发现模块隐私未同意时发生模块隐私同意前调用

1.3.5 分模块权限UI（建设中）

分模块权限是基于视频的权限组件实现。从原理上可以分为两种方案：

1、在权限组件的接口上添加模块参数，并根据模块内容弹窗模块相关代码。

2、不修改权限组件的接口，额外打点模块参数，内部在走权限UI时，根据打点的情况弹窗现实权限UI内容